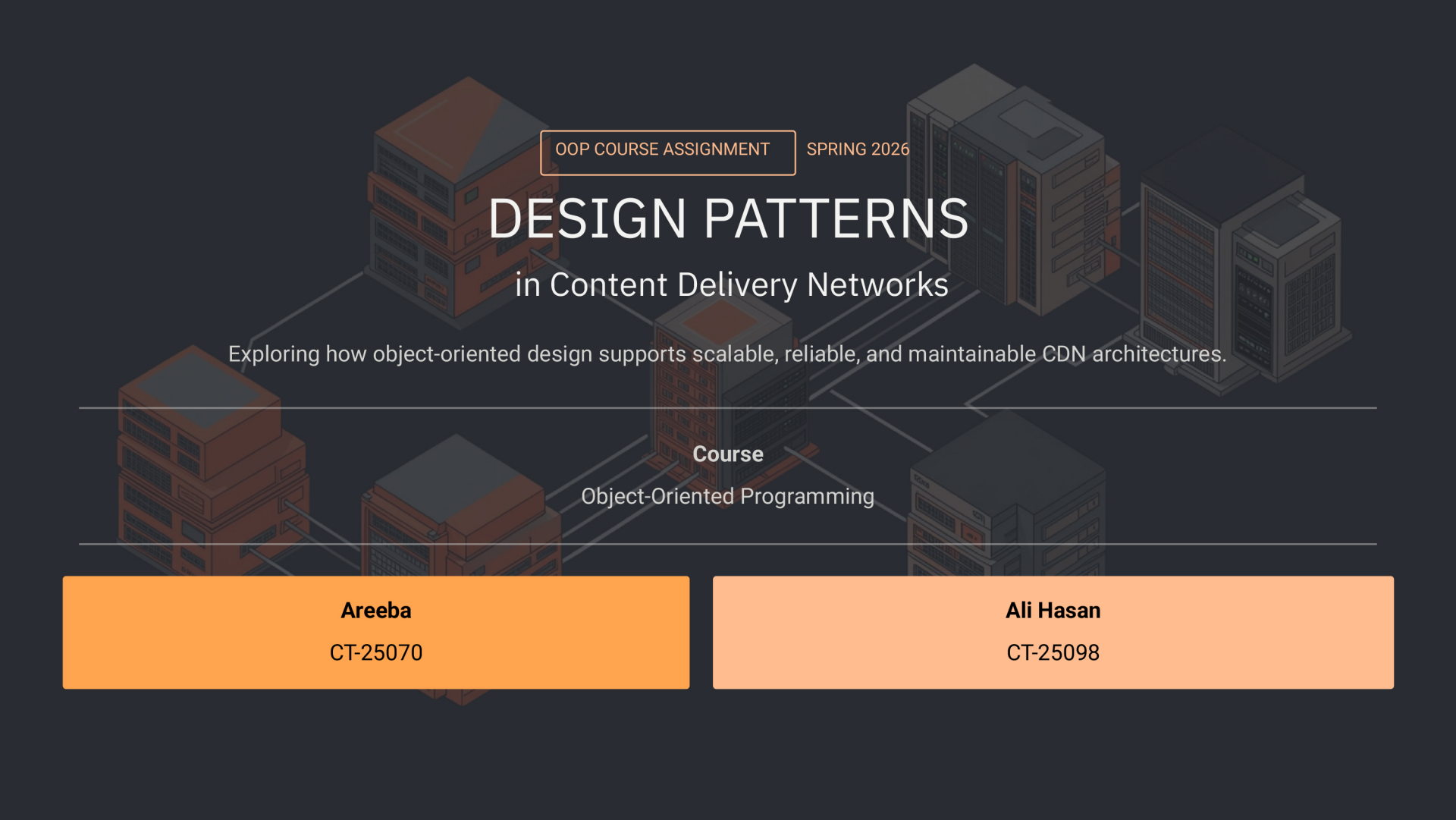


OOP COURSE ASSIGNMENT SPRING 2026

# DESIGN PATTERNS

## in Content Delivery Networks

Exploring how object-oriented design supports scalable, reliable, and maintainable CDN architectures.

**Course**

Object-Oriented Programming


**Areeba**

CT-25070

**Ali Hasan**

CT-25098

# CDN Design Patterns

How a Content Delivery Network maps perfectly onto classic Gang of Four design patterns — Proxy, Observer, Iterator, Strategy.

SOFTWARE DESIGN · DISTRIBUTED SYSTEMS



# What Is a CDN?

A **Content Delivery Network** is a globally distributed system of servers whose sole purpose is to deliver content to users as fast as possible — by moving it physically closer to them.

### Origin Server

Stores the single authoritative copy of all content.

### Edge Server

Copies deployed worldwide, close to users.

### Cache

Temporary storage on each edge server for fast retrieval.

### Request Router

The brain — routes every user request to the best edge server.



# Why a CDN?



## Real-World Scale

Every video you stream, every website you open — there is almost certainly a CDN behind it. Cloudflare and Akamai serve millions of websites, including YouTube and Netflix.

## Why Not a Food Delivery App?

A CDN is uniquely pattern-rich. Caching *is* a Proxy problem. Global config consistency *is* a Singleton problem. Reactive cache invalidation *is* an Observer problem. The patterns and the application fit together as if CDNs were designed around GoF.

# Four Design Patterns in CDNs

How classic design patterns show up in CDN architecture.



## Proxy

Edge Server shields and accelerates access to the Origin — transparent to the user.



## Observer

Origin pushes cache-invalidation events to all registered Edge Servers automatically.



## Strategy

Routing algorithms are interchangeable classes — swap without touching the router.



## Iterator

Traverse cached content across distributed edge servers without exposing internal structure.

PATTERN 1

# Iterator — Traversing Distributed Cache

## How It Works

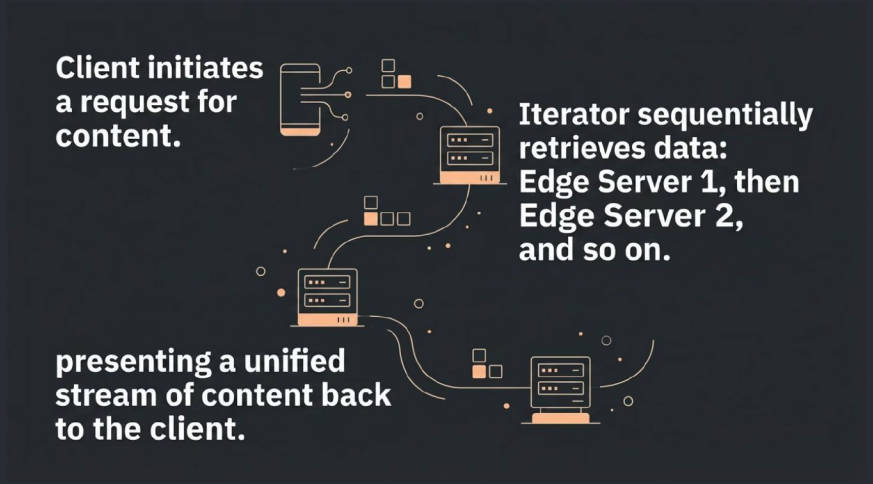
The Iterator pattern provides a standardized way to access elements of a collection sequentially without exposing its internal representation. It decouples the traversal algorithm from the collection itself.

## In CDN Context

Cached content is physically distributed across numerous edge servers worldwide. An Iterator can abstract this distribution, enabling clients to logically "step through" all cached resources as if they were a single, unified list.

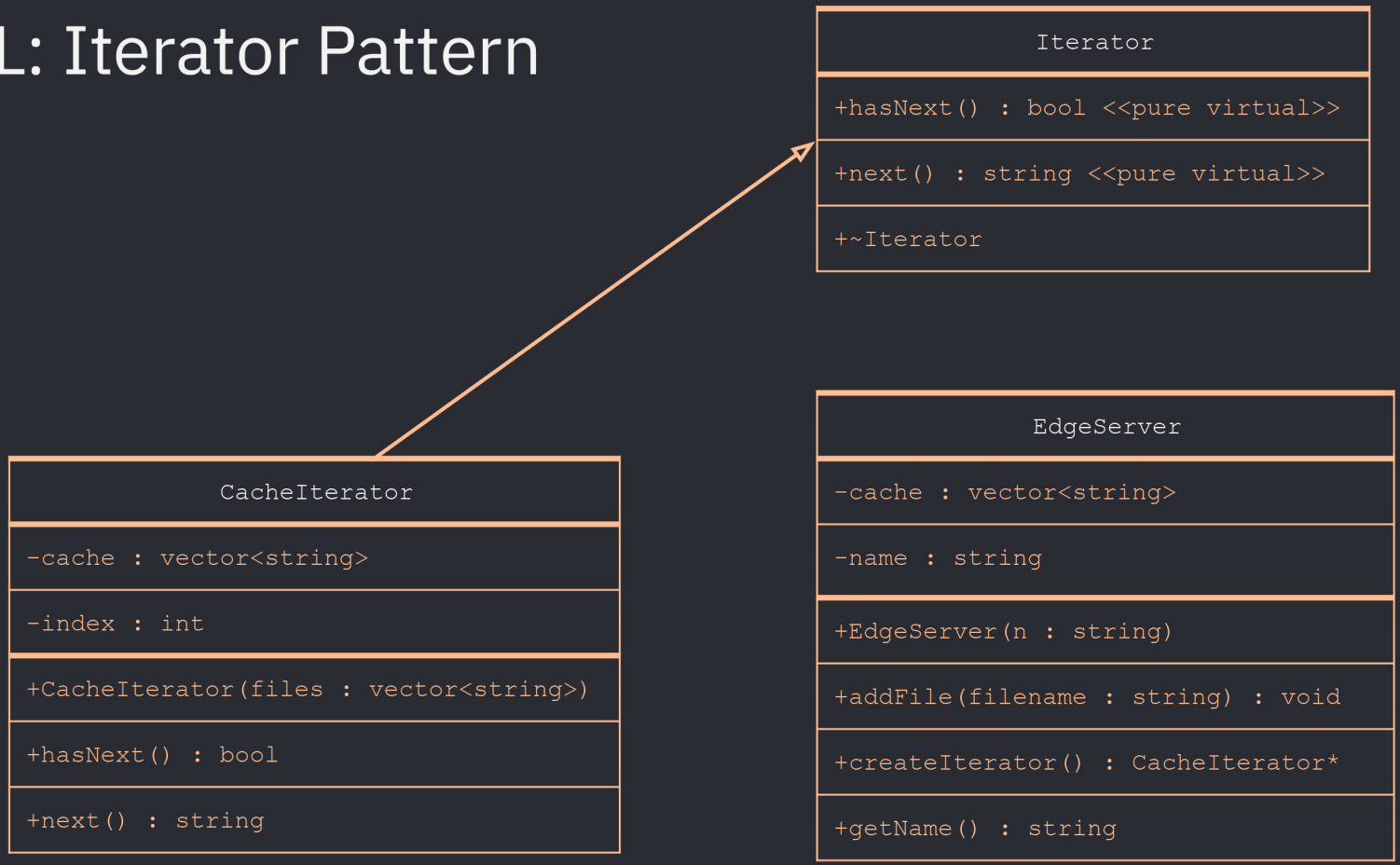
## Why It Matters

Clients, such as a content management system or a cache validation service, can iterate through resources across the entire CDN. They don't need to know the specific location or internal caching mechanisms of individual edge servers.



PATTERN 1 — UML

# UML: Iterator Pattern



PATTERN 1 — CODE

# Iterator: C++ Implementation

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Iterator{
    public:
        virtual bool hasNext() = 0;
        virtual string next() = 0;
        virtual ~Iterator() {}
};

class CacheIterator : public Iterator{
    private:
        vector<string> cache;
        int index;
    public:
        CacheIterator(vector<string> files){
            cache = files;
            index = 0;
        }
        bool hasNext() override{
            return index < cache.size();
        }
        string next() override{
            return cache[index++];
        }
};
```

```cpp
class EdgeServer{
    private:
        vector<string> cache;
        string name;
    public:
        EdgeServer(string n){
            name = n;
        }
        void addFile(string filename){
            cache.push_back(filename);
            cout << name << " added files: " << filename << endl;
        }
        Iterator* creatIterator(){
            return new CacheIterator(cache);
        }
        string getname(){
            return name;
        }
};
int main(){
    EdgeServer edge("Edge-Server-1");
    edge.addFile("video.mp4");
    edge.addFile("image.jpg");
    edge.addFile("style.css");
    edge.addFile("script.js");
    cout << "\nIterating through cached files: \n";
    Iterator* it = edge.creatIterator();
    while(it->hasNext()){
        cout << "File: " << it->next() << endl;
    }
    delete it;
    return 0;
}
```

PATTERN 2

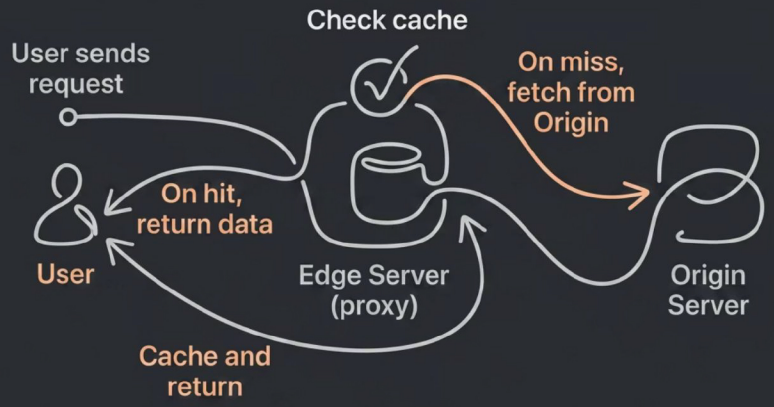
# Proxy — Edge Server as Intermediary

## How It Works

The user believes they are talking directly to the Origin Server. In reality, every request is intercepted by the **Edge Server** — the proxy.

- **Cache Hit:** File found locally → returned immediately to the user.
- **Cache Miss:** Edge fetches from Origin, stores a copy, then responds.

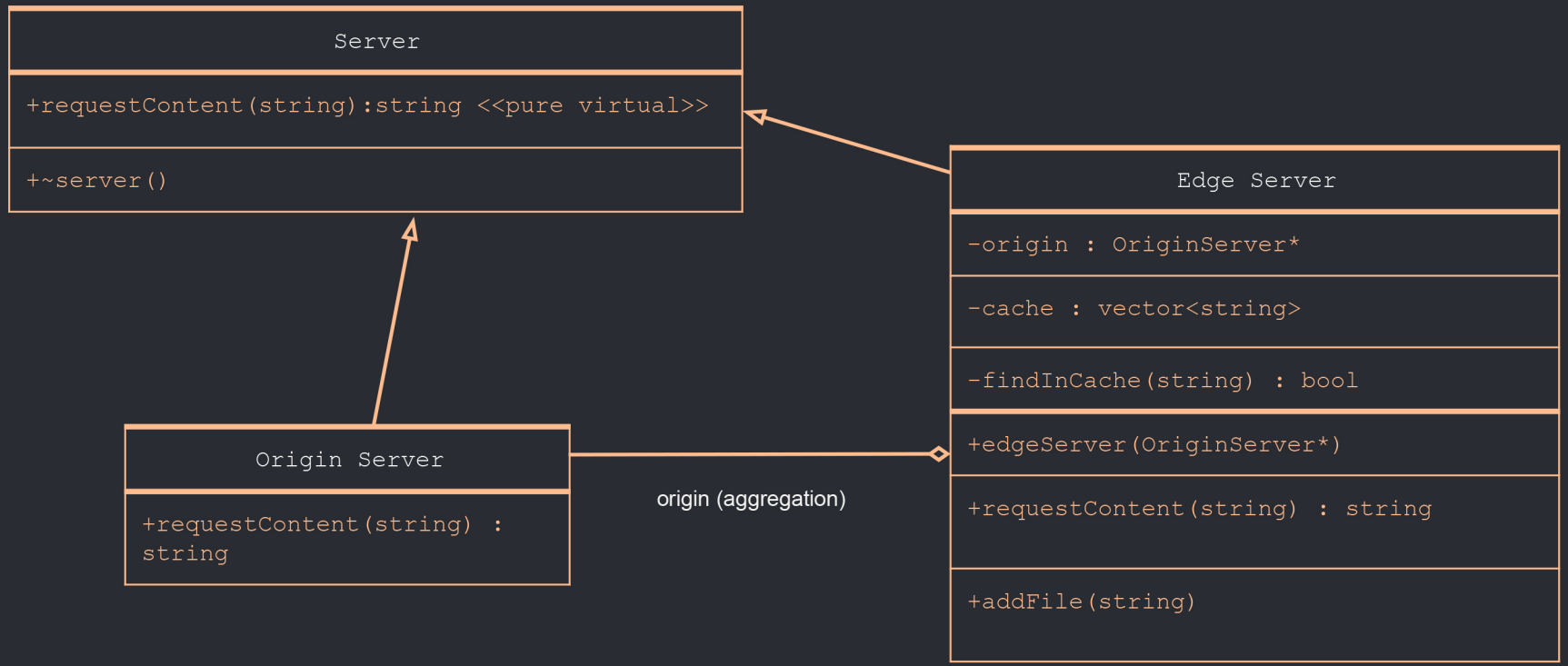
> Without Proxy, every request travels to the Origin — adding latency and crushing it under load.




**OOP Principles reinforced:** Abstraction · Encapsulation · Inheritance

PATTERN 2 — UML

# UML: Proxy Pattern



PATTERN 2— CODE

# Proxy: C++ Implementation

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Server{
    public:
        virtual string requestContent(string fileName) = 0;
        virtual ~Server(){}
};
class OriginServer : public Server{
    public:
        string requestContent(string filename) override {
            return "Data of (Origin Server)" + filename;
        }
};

class EdgeServer : public Server{
    private:
        OriginServer* origin;
        vector<string> cache;

        bool findInCache(string filename){
            for(int i=0; i<cache.size(); i++){
                if(cache[i] == filename){
                    return true;
                }
            }
            return false;
        }
    public:
```

```cpp
    public:
        EdgeServer(OriginServer* o){
            origin = o;
        }
        string requestContent(string filename){
            if(findInCache(filename)){
                return "Data from Edge Cache: " + filename;
            }
            else {
                cache.push_back(filename);
                return origin->requestContent(filename);
            }
        }
        void addFile(string filename){
            cache.push_back(filename);
        }
};
int main() {
    OriginServer origin;
    EdgeServer edge(&origin);

    edge.addFile("file1.txt");
    edge.addFile("file2.txt");

    cout << edge.requestContent("file1.txt") << endl;
    cout << edge.requestContent("file3.txt") << endl;
    cout << edge.requestContent("file2.txt") << endl;
    cout << edge.requestContent("file3.txt") << endl;

    return 0;
}
```

The `EdgeServer` wraps the `OriginServer` behind the shared `Server` interface — classic Proxy structure. The caller never needs to know which server actually responded.



PATTERN 3

# Observer — Cache Invalidation on Update

## Subject → Observers

The **Origin Server** maintains a list of every Edge Server that has cached its files. When a file changes, the Origin automatically notifies all registered Edge Servers to delete the stale copy and re-cache the new one.

New Edge Server spun up? It registers itself and immediately receives future notifications — zero manual wiring needed.

## Why Not Poll?

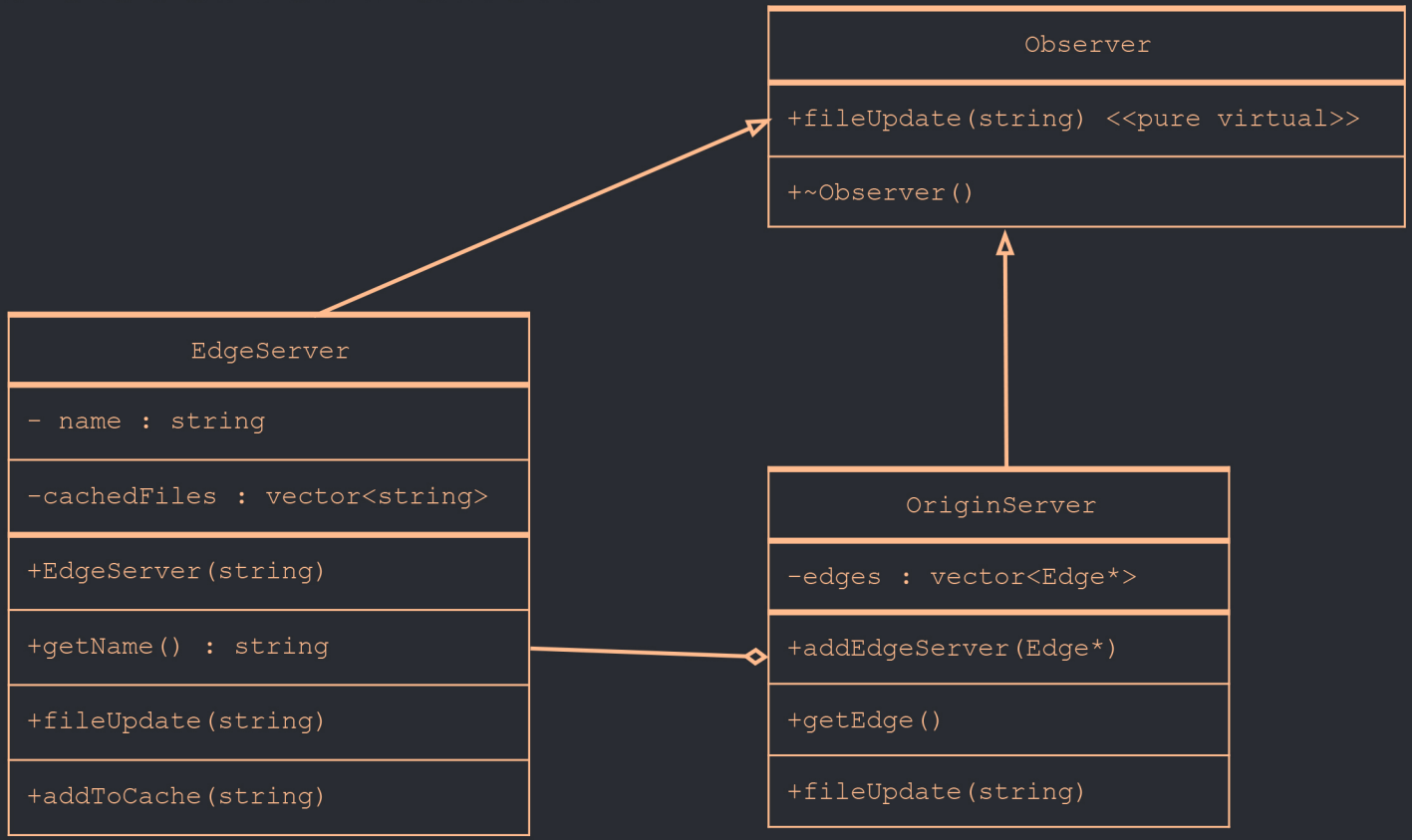
Edge Servers could constantly ask "has anything changed?" — but that wastes bandwidth and adds load. Observer **pushes** updates only when they happen, keeping the network lean and consistent.

**OOP Principles reinforced:**

Polymorphism · Abstraction · Inheritance

PATTERN 3 — UML

# UML: Observer Pattern



PATTERN 4

# Strategy — Swappable Routing Algorithms

## How It Works

The Strategy pattern lets a client hold one algorithm interface and swap implementations at runtime. In a CDN, the Request Router delegates routing decisions to the selected strategy.

## CDN Strategies

- Geographic Proximity
- Least Loaded

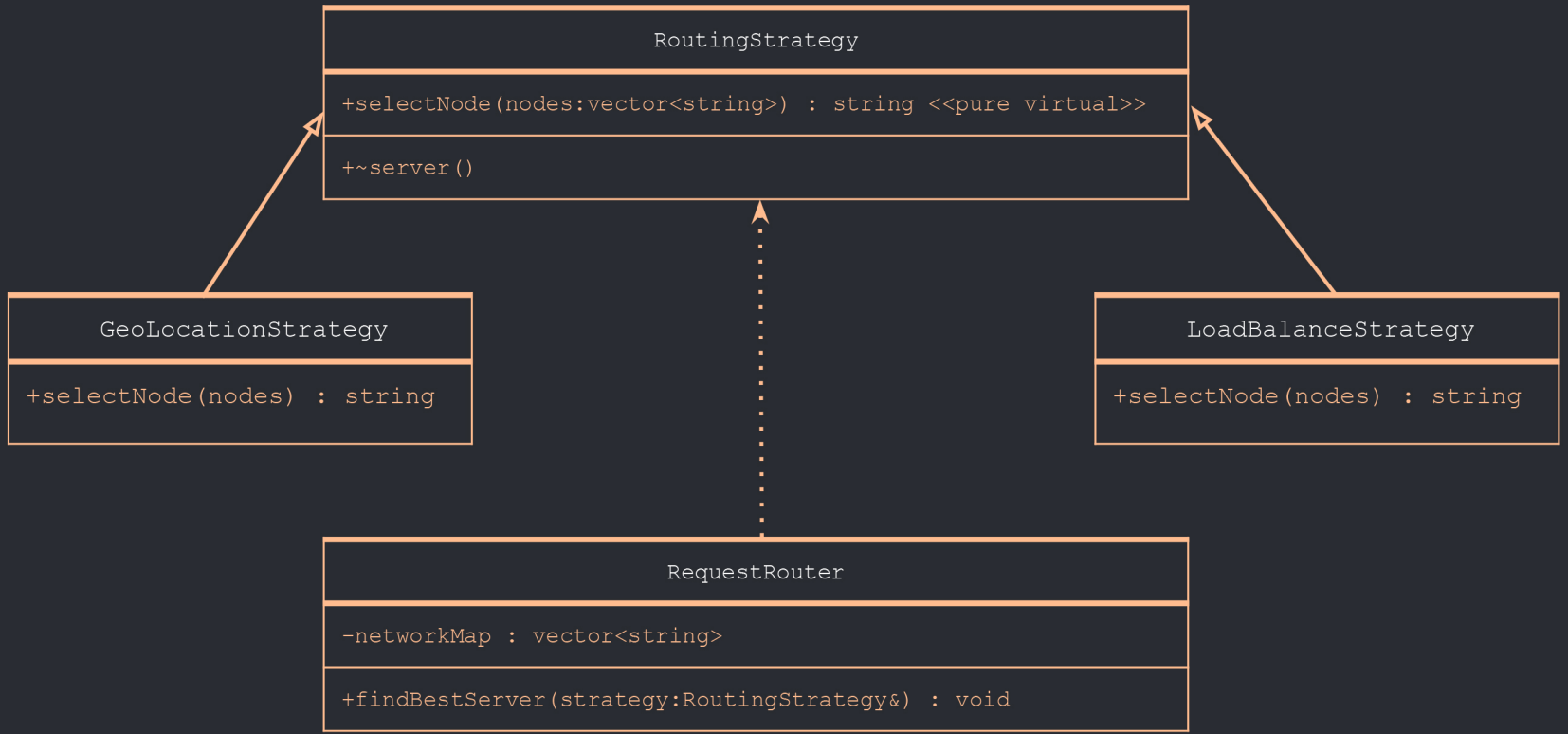
## Why It Matters

New routing logic can be added without changing the router itself. That keeps the system flexible, testable, and easy to evolve.



PATTERN 4 — UML

# UML: Strategy Pattern



# Design Patterns in CDNs: Summary

These five classic design patterns form the backbone of modern Content Delivery Networks, enabling efficiency, scalability, and robust performance.



### Iterator

Identified by traversal across distributed cache entries, nodes, or assets in a predictable sequence.



### Proxy

Identified by an edge server acting as an intermediary between users and the origin server.



### Observer

Identified by cache invalidation or update notifications that propagate when content changes.



### Strategy

Enables flexible, runtime substitution of routing algorithms to handle varying network conditions like congestion or latency.

## Why Understanding Them Matters

By abstracting complex operations, managing distributed state, and providing flexible component interaction, these patterns are critical for building systems that can handle massive scale, ensure high availability, and evolve gracefully in dynamic environments like CDNs. They exemplify how foundational software engineering principles translate into practical, high-performance distributed architectures.